Homework #19

Kevin Fang

September 20, 2025

Question 1.

The UNIX $fork()$ function is a fundamental system call used in network programming and process management. It is designed to create a new process by duplicating the existing process, known as the parent process. This new process is referred to as the child process. The $fork()$ function is essential in environments where threading support is limited or unavailable, as it allows for concurrent execution of processes.

When a program calls $fork()$, the operating system performs several key actions:

1. Process Duplication: The $fork()$ call results in the creation of a child process that is an exact copy of the parent process. This includes duplicating the process's memory space, file descriptors, and execution context.
2. Copy-on-Write Mechanism: To optimize performance and resource usage, UNIX systems often use a "copy-on-write" strategy. This means that the memory pages of the parent process are not immediately copied for the child. Instead, both processes share the same memory pages marked as read-only. Actual copying occurs only when either process attempts to modify a shared page.
3. Separate Process IDs: The child process receives a unique process identifier (PID) distinct from the parent. This PID is used to manage and control the child process independently.

Process States After Fork
After invoking $fork()$, both the parent and child processes enter specific states within the operating system:

1. Running State: Immediately after the fork, both processes are in the running state. They execute the next instruction following the $fork()$ call. The return value of $fork()$ helps distinguish between the parent and child:
   - The parent receives the child's PID as the return value.
   - The child receives a return value of zero.
2. Ready State: If the CPU is not available, either process may enter the ready state, waiting for CPU time to continue execution.
3. Blocked State: Processes can enter a blocked state if they are waiting for an event, such as I/O completion. This state is independent of the $fork()$ operation but is common in process lifecycle management.
4. Terminated State: Eventually, processes will complete their execution and enter the terminated state. The parent process can use system calls like $wait()$ to retrieve the termination status of the child process, ensuring proper cleanup.

Windows does not implement the UNIX-style $fork()$ system call. The concept and mechanism of $fork()$—duplicating a running process, including its memory and execution context—are specific to UNIX and UNIX-like operating systems (such as Linux and macOS). Windows uses a different model for creating new processes:

1. CreateProcess API: The primary way to create a new process in Windows is via the $CreateProcess()$ function. This API starts a new process from an executable file, rather than duplicating the current process.
2. No Memory Duplication: Unlike $fork()$, $CreateProcess()$ does not copy the parent's memory space, file descriptors, or execution state. Instead, it loads a fresh instance of the specified program.
3. Explicit Inheritance: If you want the child process to inherit certain handles or environment variables, you must specify this explicitly when calling $CreateProcess()$.

1. UNIX ($fork()$):
   - Duplicates the current process.
   - Both parent and child continue executing from the next instruction.
   - Useful for servers that spawn child processes to handle requests.
2. Windows ($CreateProcess()$):
   - Starts a new process from a given executable.
   - Parent and child do not share memory or execution context unless explicitly set up.
   - Concurrency Models: Windows encourages the use of threads (via $CreateThread()$) and asynchronous I/O for concurrency, rather than process duplication.